一括ファイルアップロードダイアログ UI仕様書

1. 概要

1.1 コンポーネントの目的

一括ファイルアップロードダイアログは、プロジェクト管理画面から複数のファイルやフォルダを一括でS3にアップロードするためのモーダルダイアログです。単一ファイル、一括ファイル、フォルダの3つのアップロードモードをサポートし、進捗表示やエラーハンドリングを提供します。

1.2 使用場面

- プロジェクト管理画面 → 「ファイル管理」タブ → 「一括アップロード」ボタンクリック時に表示
- モーダル形式で画面中央に表示

1.3 アクセス権限

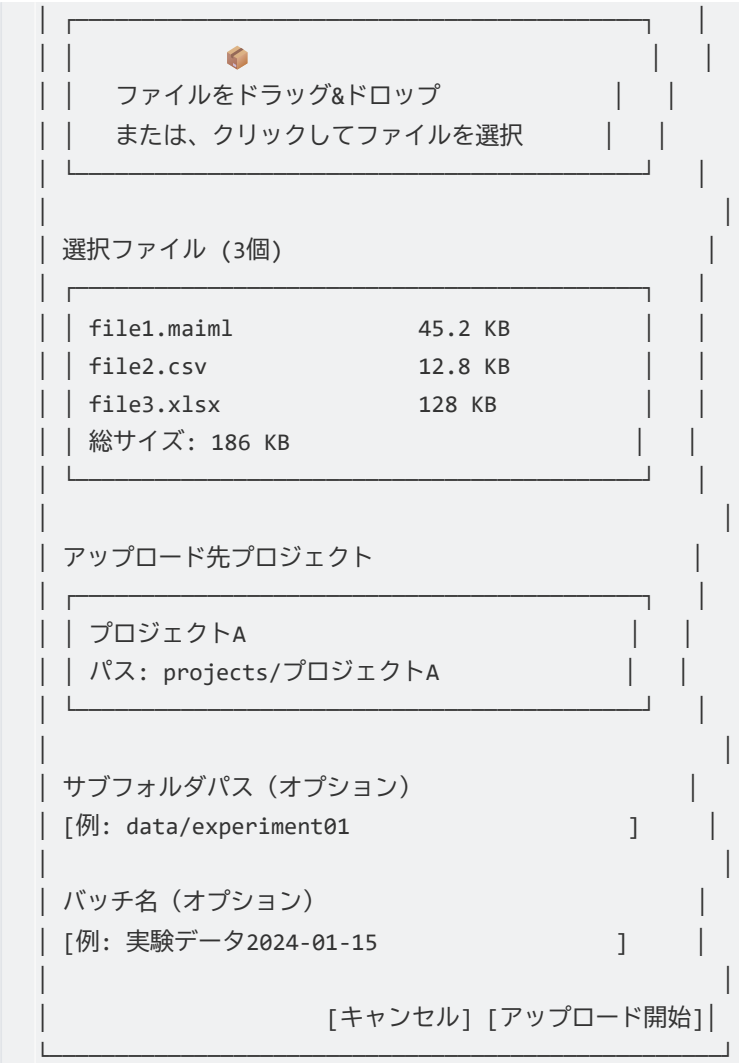
- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証
- **組織分離**: マルチテナント（organizationId）

---

2. 画面構成

2.1 レイアウト構造

```
┌──────────────────────────────────────────────────┐
│ 一括ファイルアップロード                    [×]  │
├──────────────────────────────────────────────────┤
│                                                  │
│ アップロードタイプ                               │
│ [単一ファイル] [一括ファイル] [フォルダ]         │
│                                                  │
```



2.2 主要UI要素

2.2.1 アップロードタイプセグメント

- Segmented（ブロック表示）
- 3つのオプション:
    1. **単一ファイル** (FileOutlined)
    2. **一括ファイル** (InboxOutlined)
    3. **フォルダ** (FolderOutlined)

2.2.2 ファイル選択エリア

単一ファイル / 一括ファイルモード:

- Dragger（ドラッグ&ドロップエリア）
- アイコン: InboxOutlined（青色、48px）
- 許可拡張子: `.maiml, .xml, .json, .txt, .csv, .png, .jpg, .jpeg, .gif, .pdf, .tif, .tiff, .bmp, .svg, .webp`

フォルダモード:

- ボタン:「フォルダを選択」（FolderOutlined）
- 非表示のfile input（webkitdirectory属性）

2.2.3 選択ファイル表示エリア

- 背景色: `#f5f5f5`
- 最大10ファイル表示（それ以上は「他 X ファイル...」と表示）
- 各ファイル: ファイル名 + サイズ
- 総サイズ表示（青い背景）

2.2.4 アップロード先プロジェクト表示

- 背景色: `#E6F7FC`
- ボーダー: `1px solid #0093CF`
- プロジェクト名 + パス表示

2.2.5 オプション入力フィールド

- **サブフォルダパス**: Input（プレースホルダー: "例: data/experiment01"）
- **バッチ名**: Input（プレースホルダー: "例: 実験データ2024-01-15"）

2.2.6 進捗表示

- Alert（info）: アップロード中
- Progress: 進捗バー
- LoadingOutlined アイコン

2.2.7 完了/エラー表示

- Alert（success）：アップロード完了（CheckCircleOutlined）
- Alert（error）：エラー発生（CloseCircleOutlined）

---

3. 機能仕様

3.1 アップロードモード

3.1.1 単一ファイルモード

```
uploadMode: 'single'
```

- 1ファイルのみ選択可能
- 最後に選択したファイルのみ保持
- バッチ名: `"単一アップロード：{ファイル名}"`

3.1.2 一括ファイルモード

```
uploadMode: 'bulk'
```

- 複数ファイル選択可能
- ドラッグ&ドロップまたはファイル選択ダイアログ
- バッチ名: `"一括アップロード {件数}ファイル"`

3.1.3 フォルダモード

```
uploadMode: 'folder'
```

- フォルダ全体を選択
- webkitRelativePathでフォルダ階層を保持
- バッチ名: `"フォルダアップロード：{フォルダ名}"`

3.2 ファイル選択

3.2.1 ドラッグ&ドロップ

```
<Dragger
  fileList={fileList}
  onChange={handleFileChange}
  beforeUpload={beforeUpload}  // false を返して自動アップロード無効化
  multiple={uploadMode !== 'single'}
  disabled={uploadState.isUploading}
  accept=".maiml,.xml,.json,.txt,.csv,.png,..."
  showUploadList={false}
/>
```

3.2.2 フォルダ選択

```
<input
  ref={folderInputRef}
  type="file"
  webkitdirectory=""
  directory=""
  multiple
  onChange={handleFolderSelect}
  style={{ display: 'none' }}
/>
```

3.3 アップロード先パス構築

```
const getTargetPath = () => {
  if (!preselectedFolder) {
    return '';
  }
```

```
  const basePath = `projects/${preselectedFolder}`;
  const cleanSubFolder = subFolderPath.trim().replace(/^\/+|\/+$/g, '');

  return cleanSubFolder ? `${basePath}/${cleanSubFolder}` : basePath;
};
```

パス例:

- プロジェクトルート: `projects/プロジェクトA`
- サブフォルダ指定: `projects/プロジェクトA/data/experiment01`

3.4 アップロード実行

3.4.1 処理フロー

```
const handleUpload = async () => {
  // 1. バリデーション
  if (fileList.length === 0) {
    message.warning('ファイルを選択してください');
    return;
  }

  if (!preselectedFolder) {
    message.error('プロジェクトが選択されていません');
    return;
  }

  // 2. アップロード状態初期化
  setUploadState({
    isUploading: true,
    progress: 0,
    uploadedCount: 0,
    totalCount: fileList.length,
    error: null,
  });

  // 3. ファイル準備
  const files = fileList
    .map(f => {
      const originFile = f.originFileObj;
      if (!originFile) return null;
```

```
      // フォルダモードの場合、パスを含む新しいFileオブジェクトを作成
      if (uploadMode === 'folder' && f.name.includes('/')) {
        return new File([originFile], f.name, {
          type: originFile.type,
          lastModified: originFile.lastModified,
        });
      }

      return originFile as File;
    })
    .filter((f): f is File => !!f);

  // 4. API呼び出し
  const response = await uploadFiles({
    uploadMode,
    targetPath: getTargetPath(),
    files,
    metadata: {
      projectId,
      batchName: batchName || undefined,
    },
    onProgress: (progress) => {
      setUploadState(prev => ({
        ...prev,
        progress: progress.percentage,
        uploadedCount: progress.uploadedCount,
      }));
    },
  });

  // 5. 成功処理
  if (response) {
    message.success('アップロードが完了しました');
    onUploadComplete?.({
      mode: uploadMode,
      fileCount: fileList.length,
      success: true,
    });

    // 2秒後に状態リセット
    setTimeout(() => {
      setFileList([]);
      setBatchName('');
      setSubFolderPath('');
      setUploadState({ /* 完了状態 */ });
    }, 2000);
  }
};
```

3.4.2 進捗コールバック

```
onProgress: (progress) => {
  setUploadState(prev => ({
    ...prev,
    progress: progress.percentage,
    uploadedCount: progress.uploadedCount,
  }));
}
```

3.5 キャンセル処理

```
const handleClose = () => {
  if (uploadState.isUploading) {
    Modal.confirm({
      title: 'アップロード中',
      content: 'アップロード処理が進行中です。キャンセルしますか？',
      okText: 'はい',
      cancelText: 'いいえ',
      onOk: onClose,
    });
  } else {
    onClose();
  }
};
```

3.6 バッチ名の自動生成

```
// 単一ファイル
if (newFileList.length === 1) {
  setBatchName(`単一アップロード: ${newFileList[0].name}`);
}

// 一括ファイル
else if (newFileList.length > 1) {
  setBatchName(`一括アップロード ${newFileList.length}ファイル`);
}

// フォルダ
```

```
const folderName = firstFileWithPath.webkitRelativePath.split('/')[0];
setBatchName(`フォルダアップロード: ${folderName}`);
```

---

4. データ仕様

4.1 Props定義

```typescript
interface BulkUploadDialogProps {
  /** ダイアログの表示/非表示 */
  open: boolean;
  /** 閉じるハンドラ */
  onClose: () => void;
  /** アップロード完了ハンドラ */
  onUploadComplete?: (result: {
    mode: UploadMode;
    fileCount: number;
    success: boolean;
  }) => void;
  /** 事前選択されたフォルダパス */
  preselectedFolder?: string;
  /** プロジェクトID（必須） */
  projectId: string;
}
```

Props説明:

Prop名	型	必須	説明
open	`boolean`	✅	ダイアログの表示状態
onClose	`() => void`	✅	閉じるハンドラ
onUploadComplete	`(result) => void`	-	アップロード完了時のコールバック
preselectedFolder	`string`	-	事前選択されたプロジェクトフォルダ名
projectId	`string`	✅	プロジェクトID

4.2 State定義

```
const [uploadMode, setUploadMode] = useState<UploadMode>('single');
const [fileList, setFileList] = useState<UploadFile[]>([]);
const [subFolderPath, setSubFolderPath] = useState<string>('');
const [batchName, setBatchName] = useState<string>('');
const [uploadState, setUploadState] = useState<UploadState>({
  isUploading: false,
  progress: 0,
  uploadedCount: 0,
  totalCount: 0,
  error: null,
});
```

State説明:

State名	型	初期値	説明
uploadMode	`'single' | 'bulk' | 'folder'`	`'single'`	アップロードモード
fileList	`UploadFile[]`	`[]`	選択されたファイル一覧
subFolderPath	`string`	`''`	サブフォルダパス
batchName	`string`	`''`	バッチ名
uploadState	`UploadState`	初期状態	アップロード進行状況

4.3 UploadState型定義

```
interface UploadState {
  isUploading: boolean;    // アップロード中かどうか
  progress: number;        // 進捗率（0-100）
  uploadedCount: number;   // アップロード完了ファイル数
  totalCount: number;      // 総ファイル数
  error: string | null;    // エラーメッセージ
}
```

---

5. API連携

5.1 使用するAPI

#	API名	メソッド	エンドポイント	説明
1	uploadFiles	POST	`/api/v1/files/upload`	ファイル一括アップロード

5.2 API呼び出し例

```
await uploadFiles({
  uploadMode: 'bulk',
  targetPath: 'projects/プロジェクトA/data',
  files: [File, File, File],
  metadata: {
    projectId: 'proj-123',
    batchName: '実験データ2024-01-15',
  },
  onProgress: (progress) => {
    // 進捗更新
  },
});
```

Request:

- `Content-Type` : `multipart/form-data`
- `uploadMode` : アップロードモード
- `targetPath` : アップロード先パス
- `files[]` : ファイル配列
- `metadata` : JSON文字列

Response:

```
{
  success: true,
  uploadedFiles: number,
  failedFiles: number,
  results: Array<{
    fileName: string;
    success: boolean;
    s3Key: string;
  }>
}
```

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
ファイル未選択	message.warning('ファイルを選択してください')
プロジェクト未選択	message.error('プロジェクトが選択されていません')
アップロード失敗	Alert（error）でエラーメッセージ表示
ネットワークエラー	エラーメッセージ抽出して表示

6.2 エラー処理コード

```
try {
  await uploadFiles({ /* ... */ });
  message.success('アップロードが完了しました');
} catch (error) {
  const errorMessage = extractErrorMessage(error);
  setUploadState(prev => ({
    ...prev,
    isUploading: false,
    error: errorMessage,
  }));
  message.error(errorMessage);
}
```

7. 実装ファイル

7.1 メインファイル

- **パス**: `frontend/src/components/project/BulkUploadDialog.tsx`
- **行数**: 約500行

- **エクスポート**: `BulkUploadDialog`（default export）

7.2 関連ファイル

7.2.1 APIサービス

- `frontend/src/services/fileAccessService.ts` - uploadFiles()

7.2.2 ユーティリティ

- `frontend/src/utils/errorUtils.ts` - extractErrorMessage()

7.3 依存ライブラリ

- React 18.2.0
- Ant Design v6 (Modal, Segmented, Upload, Button, Input, Alert, Progress, Space, Typography, Divider, message)
- @ant-design/icons (UploadOutlined, InboxOutlined, FolderOutlined, FileOutlined, CheckCircleOutlined, CloseCircleOutlined, LoadingOutlined)

---

8. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を再作成	Cline

---

9. 補足事項

9.1 許可ファイル拡張子

```
accept=".maiml,.xml,.json,.txt,.csv,.png,.jpg,.jpeg,.gif,.pdf,.tif,.tiff,.bmp,.svg,.webp"
```

9.2 ファイルサイズフォーマット

```
const formatFileSize = (bytes: number): string => {
  if (bytes === 0) return '0 Bytes';
  const k = 1024;
  const sizes = ['Bytes', 'KB', 'MB', 'GB'];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return `${parseFloat((bytes / Math.pow(k, i)).toFixed(2))} ${sizes[i]}`;
};
```

例:

- `0` → `"0 Bytes"`
- `1024` → `"1 KB"`
- `1048576` → `"1 MB"`

9.3 モーダル設定

```
<Modal
  title="一括ファイルアップロード"
  open={open}
  onCancel={handleClose}
  width={800}
  footer={[/* カスタムフッター */]}
  styles={{
    body: { maxHeight: '70vh', overflowY: 'auto' }
  }}
/>
```

9.4 フォルダアップロードの仕組み

```
// HTML5のwebkitdirectory属性を使用
<input
  type="file"
  webkitdirectory=""
  directory=""
  multiple
/>
```

- フォルダ内の全ファイルを選択
- `webkitRelativePath` でフォルダ階層を取得
- バックエンドでパス構造を再現

9.5 進捗表示のタイミング

1. **アップロード開始**: `isUploading: true`, `progress: 0`
2. **進行中**: `onProgress` コールバックで進捗更新
3. **完了**: `progress: 100`, `isUploading: false`
4. **2秒後**: フォームリセット

9.6 UI状態遷移

```
初期状態
  ↓ ファイル選択
ファイル選択済み
  ↓ アップロード開始
アップロード中（進捗表示）
  ↓ 完了 or エラー
完了状態 or エラー状態
  ↓ 2秒後（完了時のみ）
初期状態（リセット）
```

9.7 アクセシビリティ

- ドラッグ&ドロップとクリック選択の両方をサポート
- アップロード中のボタン無効化

- 進捗状況の視覚的表示（Progress + テキスト）
- エラーメッセージの明確な表示
- キャンセル時の確認ダイアログ

9.8 パフォーマンス考慮事項

- ファイルリスト表示は最大10件に制限
- 大量ファイルの場合は「他 X ファイル...」と省略
- アップロード完了後2秒でフォームリセット（自動クリーンアップ）